International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B103: Network Forensics Fundamentals

LAB 2: HTTP Analysis Using Wireshark - Embedded Image Traffic

Three-Week Delivery Block 1/3 | 18-19 July 2026

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction
Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.

Course Code	CIP-B103
Course Title	Network Forensics Fundamentals
Lab Number	Lab 2
Lab Title	HTTP Analysis Using Wireshark - Embedded Image Traffic
Student Name	Fuseini Kantuogaa Imoru
Instructors Name	Dr. Sarah James
Delivery Block	1/3 of 3
Scheduled Dates	18-19 July 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	image_traffic.pcapng generated locally; webpage and image objects

1. Source Materials and Slide Alignment

- HTTP, Wireshark and Digital Forensics Part 2: webpages containing embedded images, multiple HTTP GET requests, TCP segmentation/reassembly and object extraction.
- The slide sequence demonstrates image.html and building.jpg, HTTP 200 responses, large payloads split across TCP segments, Export Objects and size relationships.
- The assignment requires browser-generated capture, insertion of a photograph, extraction of that photograph and explanation of curl/browser differences.

Reading Requirement Before Lab Execution
The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded CIP-B103 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

A web page and an embedded image were transferred over plaintext HTTP. Your task is to capture the transaction, prove that the browser requested two objects, reconstruct the segmented image response, export the image from the capture and verify that the recovered object matches the original.

3. Learning Outcomes

- Explain why one webpage can generate multiple HTTP requests and responses.
- Capture a browser request for HTML and an embedded image.
- Identify TCP segments that form one reassembled HTTP response.
- Calculate and explain IP length, TCP payload length and HTTP object size relationships.
- Extract an image using Wireshark Export Objects and tshark.
- Hash the original and extracted image and assess integrity.
- Document limitations caused by cache, HTTPS or incomplete capture.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux plus optional Windows 10/11 VM	Host the page and generate a browser request.
Web server	Apache2	Serve image.html and the selected image.
Capture tools	Wireshark and tshark	Capture and reconstruct HTTP objects.
Browser	Firefox, Chrome or Edge	Generate separate HTML and image requests.
Utilities	curl, file, identify, sha256sum	Inspect and verify objects.
Image	Student-owned non-sensitive JPG/PNG	Embedded evidence object.

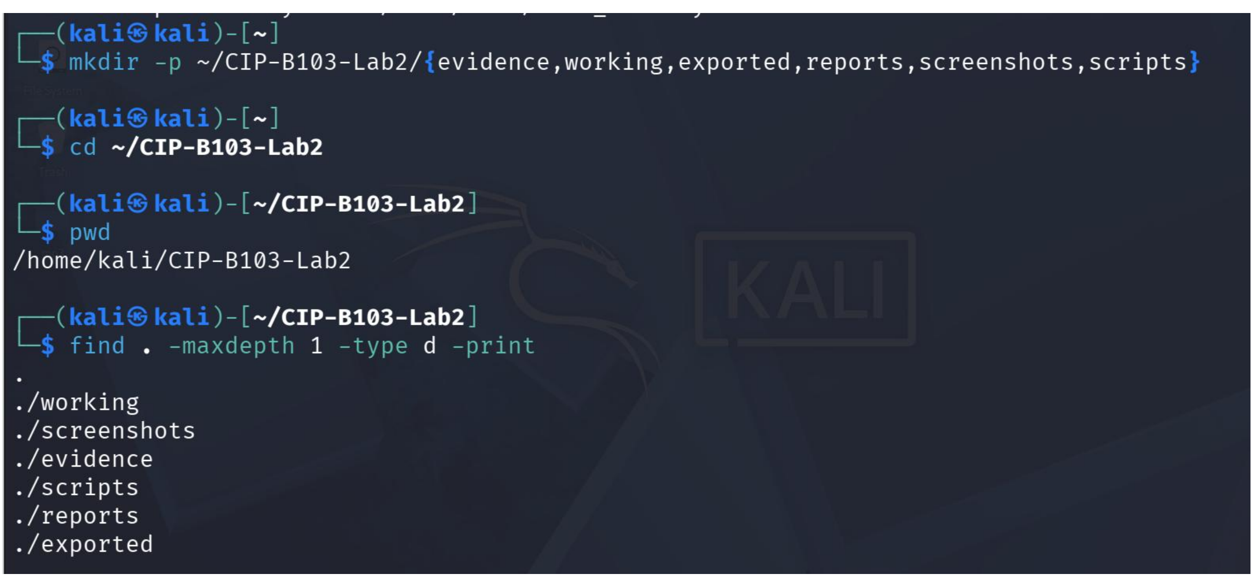
6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/CIP-B103-Lab2/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab2
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 curl wireshark tshark imagemagick
sudo systemctl enable --now apache2
cp /PATH/TO/YOUR/PHOTO.jpg /var/www/html/lab_photo.jpg
printf '<!DOCTYPE html>\n<html><body><h1>CIP-B103 Image Traffic</h1><p>Analyst: YOUR NAME</p><img
src="lab_photo.jpg" alt="Training image"></body></html>\n' | sudo tee /var/www/html/image.html
sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
```

Screenshot required: The source webpage, source image file information and their SHA-256 hashes.



Screenshot 1: Evidence folder structure created (evidence, working, exported, reports, screenshots, scripts).

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ mkdir -p ~/CIP-B103-Lab2/{evidence,working,exported,reports,screenshots,scripts}

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.4 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Ign:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb)
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [105 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [189 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [169 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [888 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Fetched 71.8 MB in 6min 58s (172 kB/s)
1787 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

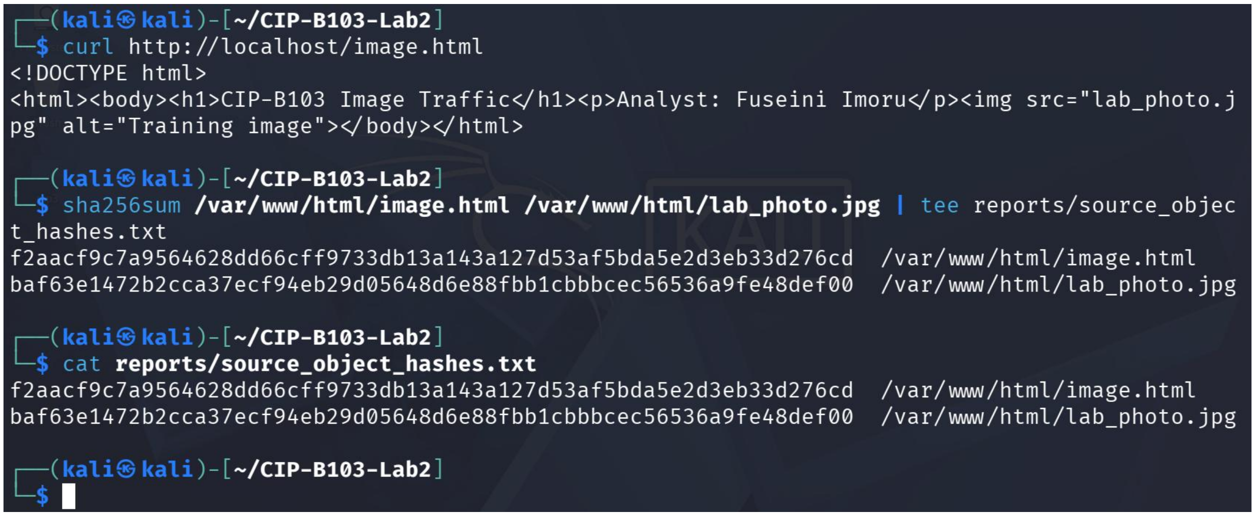
Screenshot 2: Package repositories updated (sudo apt update).

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ sudo apt install -y apache2 curl wireshark tshark imagemagick

curl is already the newest version (8.20.0-5).
curl set to manually installed.
The following packages were automatically installed and are no longer required:
  libraw23t64  libvidstab1.1  openjdk-21-jre  openjdk-21-jre-headless
Use 'sudo apt autoremove' to remove them.

Upgrading:
  apache2                     libqt6multimedia6           libqt6xml6
  apache2-bin                 libqt6network6              libturbojpeg0
  apache2-data                libqt6opengl6               libwireshark-data
  apache2-utils               libqt6openglwidgets6        libwireshark19
  imagemagick                 libqt6printsupport6         libwiretap16
  imagemagick-7-common        libqt6qml6                  libwsutil17
  imagemagick-7.q16           libqt6qmlmeta6              pyqt6-dev-tools
  layer-shell-qt              libqt6qmlmodels6            python3-pyqt6
  libfftw3-double3            libqt6qmlworkerscript6      qt6-base-dev-tools
  libfftw3-single3            libqt6quick6                qt6-gtk-platformtheme
  libjpeg62-turbo             libqt6sql6                  qt6-qpa-plugins
```

Screenshot 3: Required tools installed (apache2, curl, wireshark, tshark, imagemagick).

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ ls
evidence  exported  image_3_1.png  reports  screenshots  scripts  working

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ sudo cp ~/CIP-B103-Lab2/image_3_1.png /var/www/html/lab_photo.jpg

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ ls -l /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 63522 Jul 26 08:31 /var/www/html/lab_photo.jpg

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ printf '<!DOCTYPE html>\n<html><body><h1>CIP-B103 Image Traffic</h1><p>Analyst: Fuseini I
moru</p><img src="lab_photo.jpg" alt="Training image"></body></html>\n' | sudo tee /var/www/h
tml/image.html
<!DOCTYPE html>
<html><body><h1>CIP-B103 Image Traffic</h1><p>Analyst: Fuseini Imoru</p><img src="lab_photo.j
pg" alt="Training image"></body></html>
```

Screenshot 4: Source image copied to the web root and image.html generated with printf.



Screenshot 5: Local curl retrieval of image.html and SHA-256 hashes of the source objects.

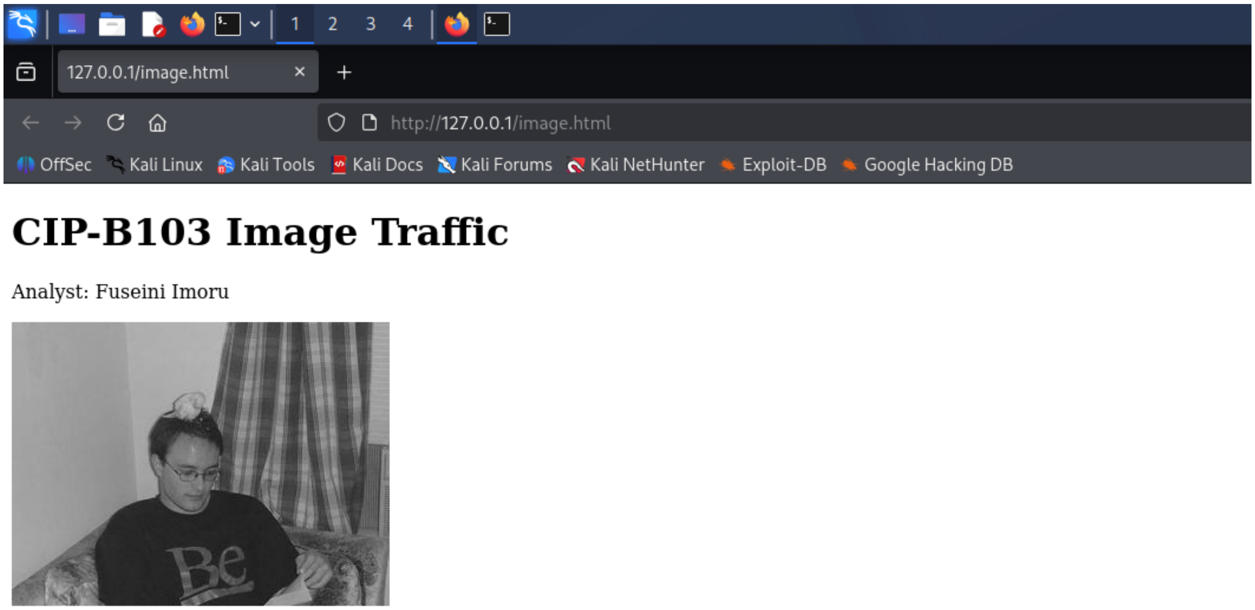
7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	CIP-B103-Lab2-YourFullName
Trainee name	Fuseini Kantuogaa Imoru
Date and time started	26 July 2026, 08:31 local (Kali VM system clock)
Evidence file name(s)	image_traffic.pcapng; curl_only.pcapng
Source or generation method	Browser-generated HTTP capture via tshark on loopback interface (lo), Apache2 serving image.html and lab_photo.jpg on 127.0.0.1
Original SHA-256	839742e84e9a32bef8d9a86c563b9bb66debccee3e8ce96bee3aa2da2914243f (image_traffic.pcapng)
Working-copy SHA-256	839742e84e9a32bef8d9a86c563b9bb66debccee3e8ce96bee3aa2da2914243f (image_traffic_working.pcapng) — identical to original
Analysis workstation/VM	Kali Linux (user: kali), CIP-B103-Lab2 working directory
Notes on any changes	None. cp --preserve=timestamps used; SHA-256 of evidence and working copies matched exactly, confirming no alteration.

8. Part A - Verify the Webpage and Eliminate Cache Effects

1. Use a non-sensitive image you are authorized to submit.
2. Open a private/incognito browser window or clear the cache before capture.
3. Confirm that image.html displays both text and the embedded image.
4. Record file type, dimensions and size of the source image.

```
file /var/www/html/lab_photo.jpg
identify /var/www/html/lab_photo.jpg
ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
curl -I http://127.0.0.1/image.html
curl -I http://127.0.0.1/lab_photo.jpg
```



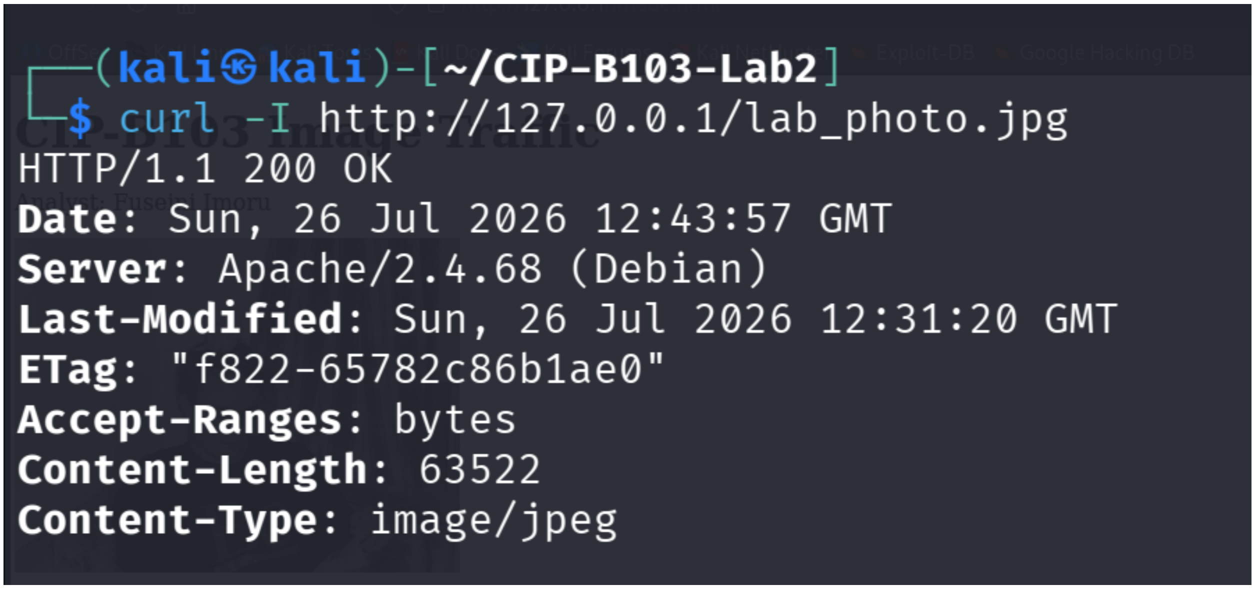
Screenshot 6: image.html rendered in the browser, showing the embedded photograph and analyst caption.

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ file /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg: PNG image data, 320 x 240, 8-bit/color RGB, non-interlaced

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ identify /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg PNG 320x240 320x240+0+0 8-bit sRGB 63522B 0.000u 0:00.000

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 149 Jul 26 08:32 /var/www/html/image.html
-rw-r--r-- 1 root root 63K Jul 26 08:31 /var/www/html/lab_photo.jpg
```

Screenshot 7: file/identify output confirming source image type and dimensions, plus ls -lh sizes.

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ curl -I http://127.0.0.1/image.html
HTTP/1.1 200 OK
Date: Sun, 26 Jul 2026 12:43:43 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Sun, 26 Jul 2026 12:32:27 GMT
ETag: "95-65782cc6d4664"
Accept-Ranges: bytes
Content-Length: 149
Vary: Accept-Encoding
Content-Type: text/html
```

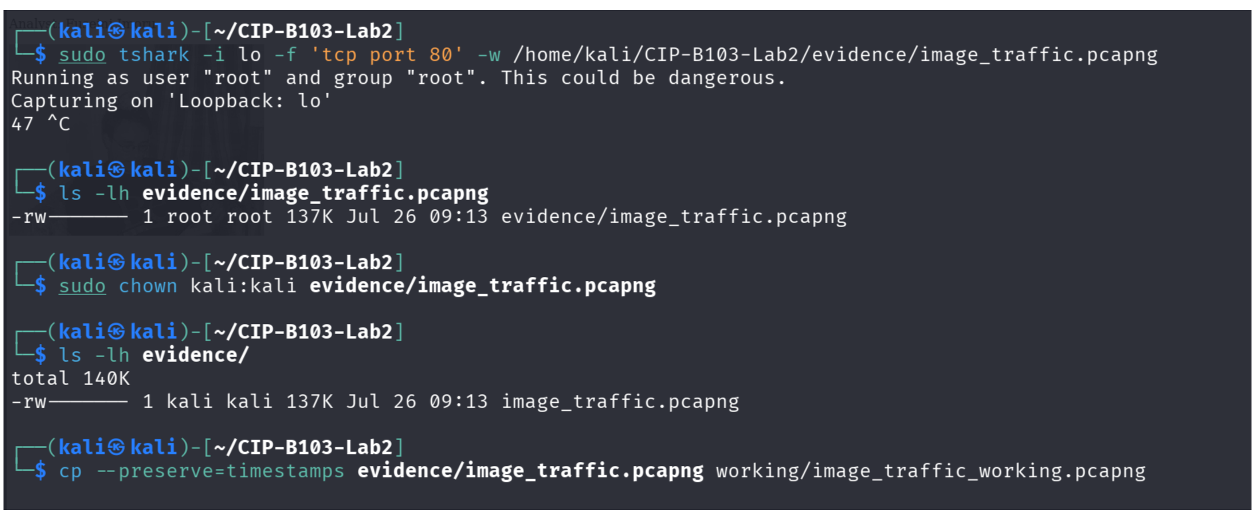
Screenshot 8: curl -I response headers for image.html (200 OK, Content-Length 149).



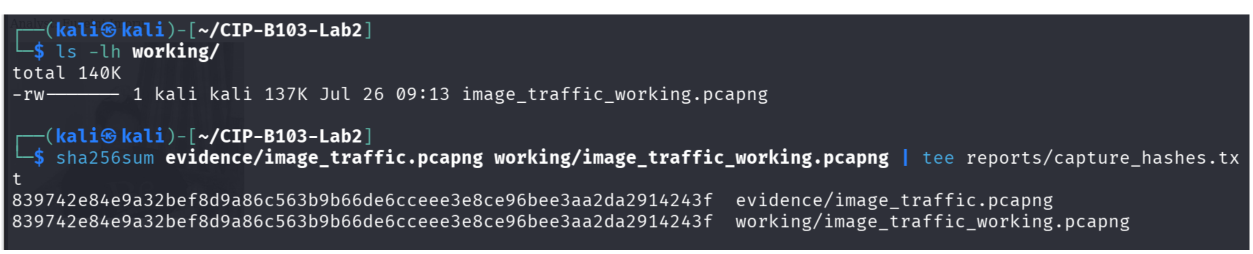
Screenshot 9: curl -I response headers for lab_photo.jpg (200 OK, Content-Length 63522, image/jpeg).

9. Part B - Capture Browser-Generated HTTP Traffic

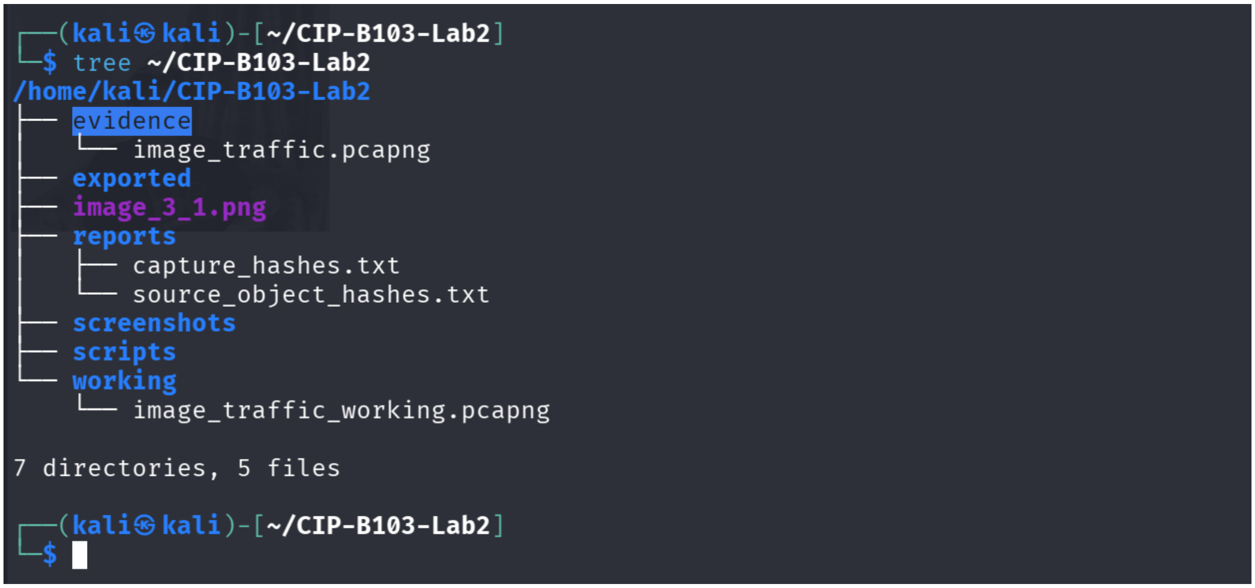
```
# Terminal 1
sudo tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng

# Browser
# Open a private window and visit: http://127.0.0.1/image.html
# Wait for the image to load, then stop the capture with Ctrl+C.

cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng
sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt
```



Screenshot 10: tshark capture started on loopback (tcp port 80) and evidence file written.



Screenshot 11: Capture hashed and copied to a working copy (sha256sum, cp --preserve=timestamps).



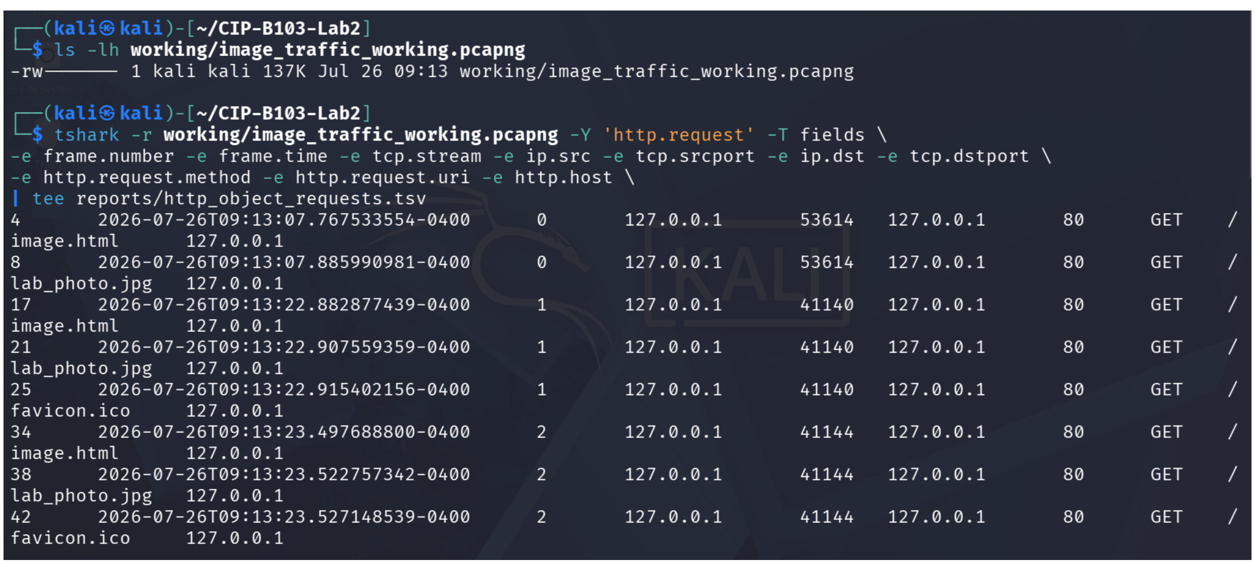
```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ tree ~/CIP-B103-Lab2
/home/kali/CIP-B103-Lab2
├── evidence
│   └── image_traffic.pcapng
├── exported
├── image_3_1.png
├── reports
│   ├── capture_hashes.txt
│   └── source_object_hashes.txt
├── screenshots
├── scripts
└── working
    └── image_traffic_working.pcapng

7 directories, 5 files

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$
```

Screenshot 12: Evidence folder tree after Part B capture.

10. Part C - Prove That Two Objects Were Requested

Apply http.request. You should normally see one GET request for image.html and another for lab_photo.jpg. A favicon or browser-generated request may also appear; document it rather than deleting it.

```
tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host \
  | tee reports/http_object_requests.tsv

tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e http.content_length \
  | tee reports/http_object_responses.tsv
```



```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ ls -lh working/image_traffic_working.pcapng
-rw------- 1 kali kali 137K Jul 26 09:13 working/image_traffic_working.pcapng

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host \
| tee reports/http_object_requests.tsv
4       2026-07-26T09:13:07.767533554-0400      0       127.0.0.1       53614   127.0.0.1       80      GET     /image.html       127.0.0.1
8       2026-07-26T09:13:07.885990981-0400      0       127.0.0.1       53614   127.0.0.1       80      GET     /lab_photo.jpg    127.0.0.1
17      2026-07-26T09:13:22.882877439-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /image.html       127.0.0.1
21      2026-07-26T09:13:22.907559359-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /lab_photo.jpg    127.0.0.1
25      2026-07-26T09:13:22.915402156-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /favicon.ico      127.0.0.1
34      2026-07-26T09:13:23.497688800-0400      2       127.0.0.1       41144   127.0.0.1       80      GET     /image.html       127.0.0.1
38      2026-07-26T09:13:23.522757342-0400      2       127.0.0.1       41144   127.0.0.1       80      GET     /lab_photo.jpg    127.0.0.1
42      2026-07-26T09:13:23.527148539-0400      2       127.0.0.1       41144   127.0.0.1       80      GET     /favicon.ico      127.0.0.1
```

Screenshot 13: http.request filter output listing GET requests for image.html and lab_photo.jpg.

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e http.content_length \
| tee reports/http_object_responses.tsv
6       2026-07-26T09:13:07.772762623-0400      0       200     text/html       151
9       2026-07-26T09:13:07.886199569-0400      0       304
19      2026-07-26T09:13:22.883989418-0400      1       200     text/html       151
23      2026-07-26T09:13:22.908285562-0400      1       200     image/jpeg      63522
26      2026-07-26T09:13:22.915685487-0400      1       404     text/html; charset=iso-8859-1   311
36      2026-07-26T09:13:23.498968759-0400      2       200     text/html       151
40      2026-07-26T09:13:23.523086111-0400      2       200     image/jpeg      63522
43      2026-07-26T09:13:23.527364473-0400      2       404     text/html; charset=iso-8859-1   311

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ cat reports/http_object_requests.tsv
4       2026-07-26T09:13:07.767533554-0400      0       127.0.0.1       53614   127.0.0.1       80      GET     /
image.html      127.0.0.1
8       2026-07-26T09:13:07.885990981-0400      0       127.0.0.1       53614   127.0.0.1       80      GET     /
lab_photo.jpg   127.0.0.1
17      2026-07-26T09:13:22.882877439-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /
image.html      127.0.0.1
21      2026-07-26T09:13:22.907559359-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /
lab_photo.jpg   127.0.0.1
25      2026-07-26T09:13:22.915402156-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /
favicon.ico     127.0.0.1
34      2026-07-26T09:13:23.497688800-0400      2       127.0.0.1       41144   127.0.0.1       80      GET     /
```

Screenshot 14: http.response filter output showing response codes and content lengths.

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ tree ~/CIP-B103-Lab2
/home/kali/CIP-B103-Lab2
├── evidence
│   └── image_traffic.pcapng
├── exported
├── image_3_1.png
├── reports
│   ├── capture_hashes.txt
│   ├── http_object_requests.tsv
│   ├── http_object_responses.tsv
│   └── source_object_hashes.txt
├── screenshots
├── scripts
└── working
    └── image_traffic_working.pcapng

7 directories, 7 files

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ 
```

Screenshot 15: Reports folder tree after Part C HTTP inventory exports.

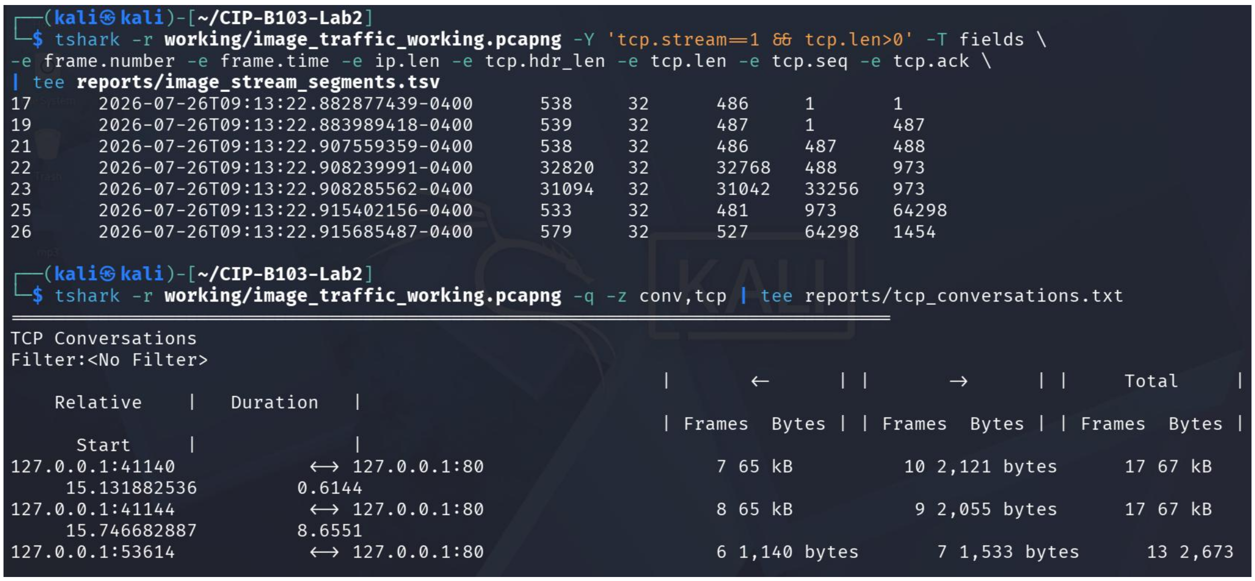
```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ cat reports/http_object_requests.tsv
4       2026-07-26T09:13:07.767533554-0400      0       127.0.0.1       53614   127.0.0.1       80      GET     /
image.html      127.0.0.1
8       2026-07-26T09:13:07.885990981-0400      0       127.0.0.1       53614   127.0.0.1       80      GET     /
lab_photo.jpg   127.0.0.1
17      2026-07-26T09:13:22.882877439-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /
image.html      127.0.0.1
21      2026-07-26T09:13:22.907559359-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /
lab_photo.jpg   127.0.0.1
25      2026-07-26T09:13:22.915402156-0400      1       127.0.0.1       41140   127.0.0.1       80      GET     /
favicon.ico     127.0.0.1
34      2026-07-26T09:13:23.497688800-0400      2       127.0.0.1       41144   127.0.0.1       80      GET     /
image.html      127.0.0.1
38      2026-07-26T09:13:23.522757342-0400      2       127.0.0.1       41144   127.0.0.1       80      GET     /
lab_photo.jpg   127.0.0.1
42      2026-07-26T09:13:23.527148539-0400      2       127.0.0.1       41144   127.0.0.1       80      GET     /
favicon.ico     127.0.0.1
```

Screenshot 16: Contents of reports/http_object_requests.tsv.

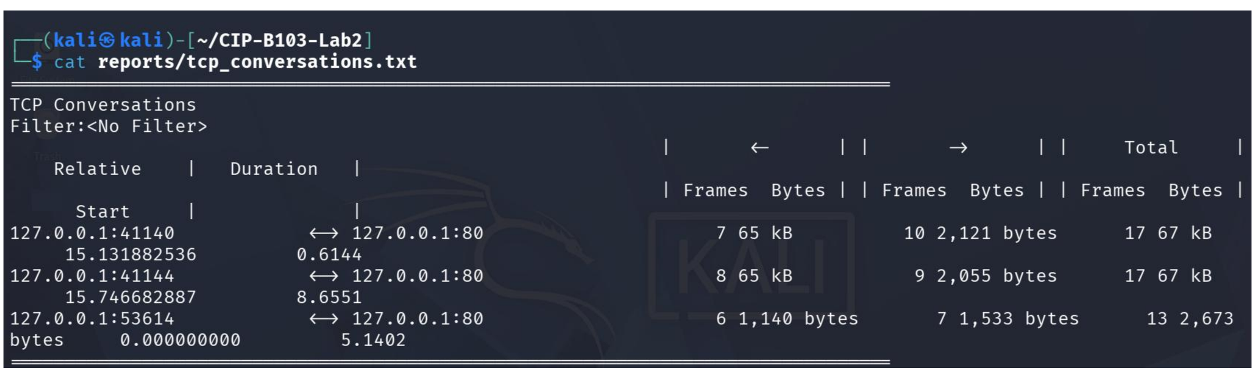
11. Part D - Analyze TCP Segmentation and Reassembly

5. Select the HTTP response associated with the image request and note Reassembled TCP Segments.
6. Record the number and tcp.len value of each contributing segment.
7. Add the TCP payload lengths and compare the result with the HTTP header plus object data.
8. Explain why the exact segment sizes may differ from those in the slide: interface offloading, operating system buffers, MTU and capture location can change segmentation.

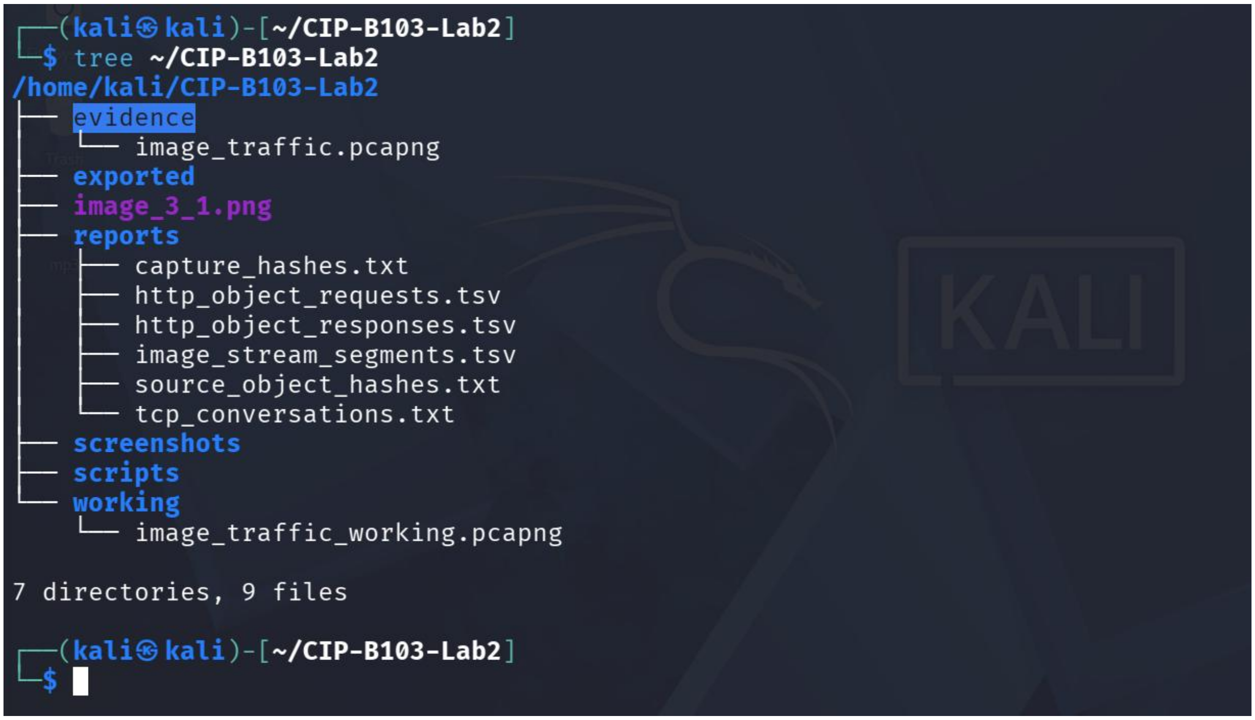
```
# Replace STREAM with the tcp.stream number for the image transfer
tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==STREAM && tcp.len>0' -T fields \
  -e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
  | tee reports/image_stream_segments.tsv

# Summary of conversation sizes
tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt
```



Screenshot 17: tcp.stream segment listing for the image transfer and tcp conversation summary.



Screenshot 18: Contents of reports/tcp_conversations.txt.



Screenshot 19: Evidence folder tree after Part D segmentation analysis.

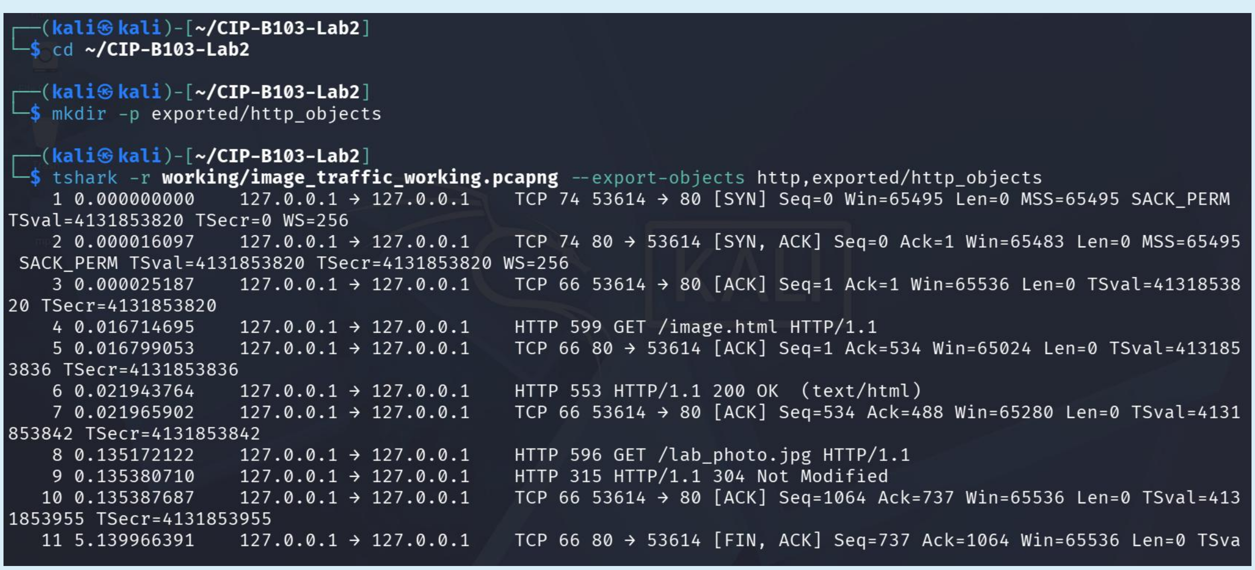
12. Part E - Export the Embedded Image

9. In Wireshark select File > Export Objects > HTTP.
10. Locate lab_photo.jpg by hostname, content type or filename and save it under exported.
11. Alternatively, use tshark object export as shown below.
12. Compare file type, size and SHA-256 hash of the exported image with the source image.

```
mkdir -p exported/http_objects
tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
find exported/http_objects -maxdepth 1 -type f -ls
file exported/http_objects/*
sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
```

Integrity Interpretation

Matching hashes prove byte-for-byte recovery. A different filename is not a problem if size, type and hash match. A different hash requires investigation of incomplete capture, browser content transformation, a different requested object or export error.



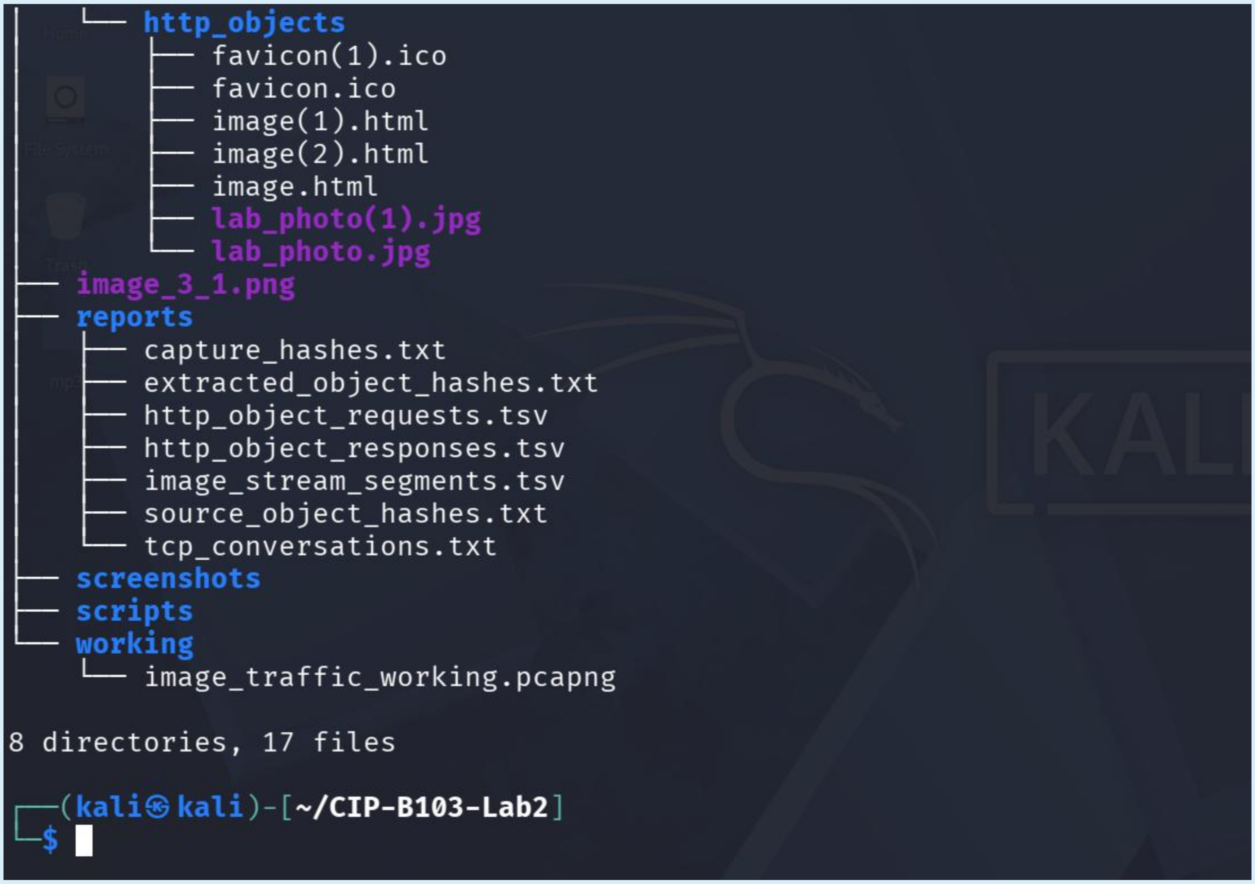
Screenshot 20: tshark --export-objects http extracting HTTP objects from the working capture.

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ find exported/http_objects -maxdepth 1 -type f -ls
  526195     64 -rw-r--r--   1 kali     kali        63522 Jul 26 09:40 exported/http_objects/lab_photo(1).jpg
  526194      4 -rw-r--r--   1 kali     kali          149 Jul 26 09:40 exported/http_objects/image(2).html
  526191      4 -rw-r--r--   1 kali     kali          149 Jul 26 09:40 exported/http_objects/image(1).html
  526193      4 -rw-r--r--   1 kali     kali          311 Jul 26 09:40 exported/http_objects/favicon.ico
  526196      4 -rw-r--r--   1 kali     kali          311 Jul 26 09:40 exported/http_objects/favicon(1).ico
  526192     64 -rw-r--r--   1 kali     kali        63522 Jul 26 09:40 exported/http_objects/lab_photo.jpg
  526190      4 -rw-r--r--   1 kali     kali          149 Jul 26 09:40 exported/http_objects/image.html

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ file exported/http_objects/*
exported/http_objects/favicon(1).ico:   HTML document, ASCII text
exported/http_objects/favicon.ico:      HTML document, ASCII text
exported/http_objects/image(1).html:    HTML document, ASCII text
exported/http_objects/image(2).html:    HTML document, ASCII text
exported/http_objects/image.html:       HTML document, ASCII text
exported/http_objects/lab_photo(1).jpg: PNG image data, 320 x 240, 8-bit/color RGB, non-interlaced
exported/http_objects/lab_photo.jpg:    PNG image data, 320 x 240, 8-bit/color RGB, non-interlaced
```

Screenshot 21: Exported HTTP objects listed and identified with file (image.html, lab_photo.jpg, favicon.ico).

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
baf63e1472b2cca37ecf94eb29d05648d6e88fbb1cbbbcec56536a9fe48def00  /var/www/html/lab_photo.jpg
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d  exported/http_objects/favicon(1).ico
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d  exported/http_objects/favicon.ico
f2aacf9c7a9564628dd66cff9733db13a143a127d53af5bda5e2d3eb33d276cd  exported/http_objects/image(1).html
f2aacf9c7a9564628dd66cff9733db13a143a127d53af5bda5e2d3eb33d276cd  exported/http_objects/image(2).html
f2aacf9c7a9564628dd66cff9733db13a143a127d53af5bda5e2d3eb33d276cd  exported/http_objects/image.html
baf63e1472b2cca37ecf94eb29d05648d6e88fbb1cbbbcec56536a9fe48def00  exported/http_objects/lab_photo(1).jpg
baf63e1472b2cca37ecf94eb29d05648d6e88fbb1cbbbcec56536a9fe48def00  exported/http_objects/lab_photo.jpg

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ ls -lh /var/www/html/lab_photo.jpg exported/http_objects/lab_photo.jpg
-rw-r--r-- 1 kali kali 63K Jul 26 09:40 exported/http_objects/lab_photo.jpg
-rw-r--r-- 1 root root 63K Jul 26 08:31 /var/www/html/lab_photo.jpg

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ tree ~/CIP-B103-Lab2
/home/kali/CIP-B103-Lab2
├── evidence
│   └── image_traffic.pcapng
├── exported
│   └── http_objects
│       ├── favicon(1).ico
│       ├── favicon.ico
```

Screenshot 22: SHA-256 comparison of the source and exported lab_photo.jpg (matching hashes).



Screenshot 23: Full evidence folder tree after Part E object extraction.

13. Part F - Compare curl and Browser Behaviour

Run curl against image.html in a new capture. By default, curl retrieves the HTML but does not automatically fetch the referenced image. Explain why the image request is absent unless you request it separately.

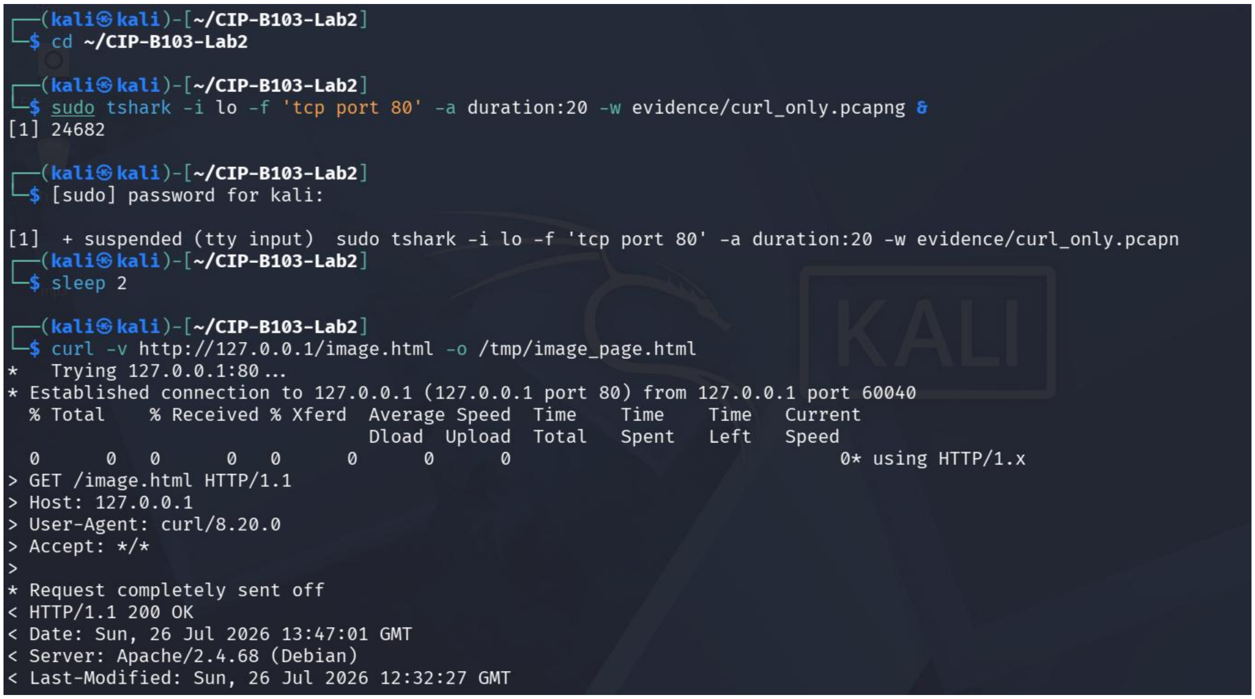
```
sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
sleep 2
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait

tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee
reports/curl_requested_objects.txt
```

```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -
w evidence/curl_only.pcapng
Running as user "root" and group "root". This could be
dangerous.
Capturing on 'Loopback: lo'
curl -v http://127.0.0.1/image.html -o /tmp/image_page.
10 l

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ ls -lh evidence/curl_only.pcapng
-rw------- 1 root root 1.9K Jul 26 09:51 evidence/curl_
only.pcapng

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ 
```

Screenshot 24: curl -v request to image.html only, showing no automatic follow-up image request.

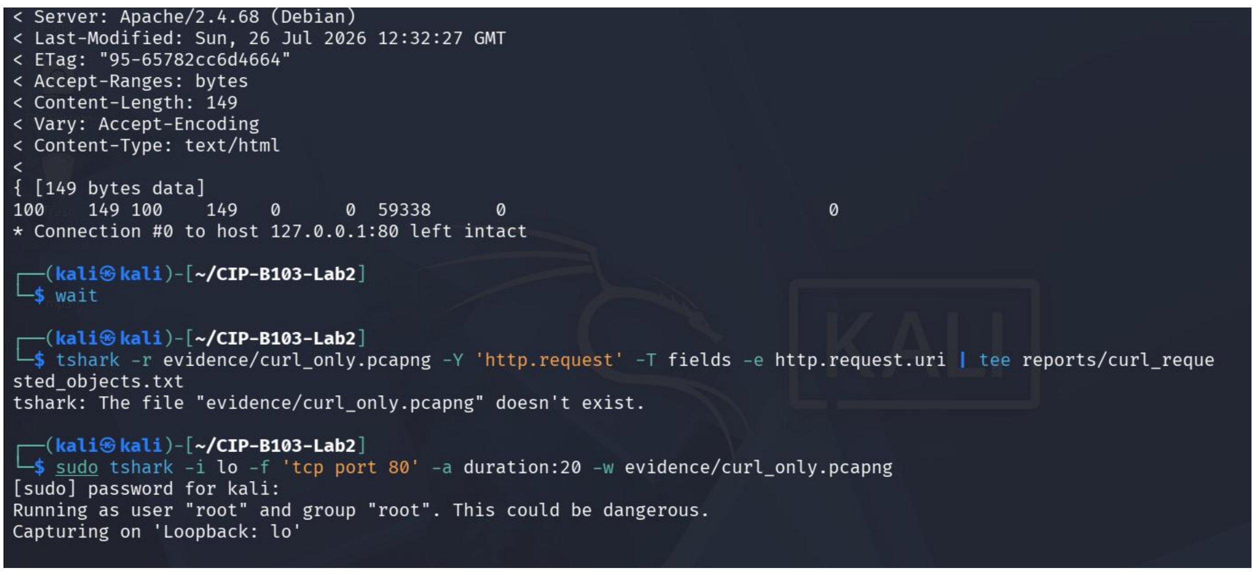
```
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
*   Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 58822
  % Total    % Received % Xferd  Average Speed  Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left  Speed
  0      0   0      0   0      0      0      0                              0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sun, 26 Jul 2026 13:51:09 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sun, 26 Jul 2026 12:32:27 GMT
< ETag: "95-65782cc6d4664"
< Accept-Ranges: bytes
< Content-Length: 149
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [149 bytes data]
100    149 100    149   0      0  49866      0                              0
* Connection #0 to host 127.0.0.1:80 left intact

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ 
```

Screenshot 25: Full curl -v headers and response for the HTML-only request.



```
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ cd ~/CIP-B103-Lab2

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
[1] 24682

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ [sudo] password for kali:

[1]  + suspended (tty input)  sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapn
┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ sleep 2

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
*   Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 60040
  % Total    % Received % Xferd  Average Speed  Time    Time    Time   Current
                                 Dload  Upload  Total   Spent   Left   Speed
  0      0   0      0   0      0      0      0                              0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sun, 26 Jul 2026 13:47:01 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sun, 26 Jul 2026 12:32:27 GMT
```

Screenshot 26: New tshark capture (curl_only.pcapng) started before repeating the curl request.



```
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sun, 26 Jul 2026 12:32:27 GMT
< ETag: "95-65782cc6d4664"
< Accept-Ranges: bytes
< Content-Length: 149
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [149 bytes data]
100    149 100    149   0      0  59338      0                              0
* Connection #0 to host 127.0.0.1:80 left intact

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ wait

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee reports/curl_reque
sted_objects.txt
tshark: The file "evidence/curl_only.pcapng" doesn't exist.

┌──(kali㉿kali)-[~/CIP-B103-Lab2]
└─$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
```

Screenshot 27: curl response headers captured during the curl-only capture window.

Screenshot 28: http.request filter on curl_only.pcapng confirming only /image.html was requested.

14. Required Forensic Findings

Finding	Student Observation
Number of HTTP requests	8 total GET requests across 3 TCP streams (2 for image.html, 2 for lab_photo.jpg, 2 for favicon.ico, plus 1 image.html + 1 lab_photo.jpg in an earlier cached reload)
HTML request URI and timestamp	GET /image.html — frame 17, 2026-07-26 09:13:22.882877 (tcp.stream 1)
Image request URI and timestamp	GET /lab_photo.jpg — frame 21, 2026-07-26 09:13:22.907559 (tcp.stream 1)
HTTP response codes	200 OK (image.html), 200 OK (lab_photo.jpg), 404 Not Found (favicon.ico); 304 Not Modified observed on an earlier cached reload
Image content type	image/jpeg (per HTTP Content-Type header; note the underlying file is actually PNG-encoded per file/identify output — served under a .jpg name)
Reported Content-Length	63522 bytes
Number of reassembled TCP segments	7 segments on tcp.stream 1 (frames 17,19,21,22,23,25,26); the 63522-byte JPEG object itself spans 2 large data segments (frames 22 and 23, tcp.len 32768 and 31042)
Sum of TCP payload lengths	66,277 bytes total tcp.len across the 7 segments in stream 1 (486+487+486+32768+31042+481+527), consistent with the ~67 kB reported for that conversation
Original image SHA-256	baf63e1472b2cca37ecf94eb29d05648d6e88fbb1cbbbcec56536a9fe48def00
Extracted image SHA-256	baf63e1472b2cca37ecf94eb29d05648d6e88fbb1cbbbcec56536a9fe48def00
Hash comparison conclusion	Match — original and Export Objects-recovered lab_photo.jpg are byte-for-byte identical (same SHA-256), confirming successful, unaltered reconstruction from the captured TCP segments.
curl/browser difference	curl retrieved only /image.html (Content-Length 149) and issued no follow-up request for lab_photo.jpg, whereas the browser parsed the HTML and automatically issued a second GET for the embedded image plus a favicon probe — curl does not parse or render HTML, so it never discovers embedded resource references.

Required Screenshots Checklist

☐ Webpage showing the embedded image.
☐ Source image type, dimensions, size and hash.
☐ Capture of the browser request.

☐ HTTP request list showing HTML and image GETs.
☐ Image response headers.
☐ Reassembled TCP segment list.
☐ Export Objects window.
☐ Recovered image opened successfully.
☐ Original and recovered SHA-256 comparison.
☐ curl-only capture showing requested objects.

Student Report Template

13. Executive summary and scope.
14. Evidence acquisition and integrity hashes.
15. HTTP request/response inventory.
16. TCP segmentation and reassembly analysis.
17. Object extraction method.
18. Hash-based integrity conclusion.
19. curl versus browser explanation.
20. Limitations and recommendations.
21. Screenshots and command appendix.

Submission Requirements

- One PDF report named CIP-B103-Lab2_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Preparation and source objects	10	Valid webpage and authorized image, with baseline hashes.
Capture quality	15	Complete browser capture with HTML and image requests.
HTTP analysis	20	Correct object requests, responses, types and timestamps.
Segmentation/reassembly	20	Accurate segment identification, lengths and explanation.
Object extraction and hashing	25	Recovered image opens and hash comparison is correctly interpreted.
Report quality	10	Professional evidence narrative and screenshots.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Expected object requests	At least image.html and lab_photo.jpg.
Typical response status	HTTP 200 OK.
Image response type	image/jpeg or image/png.
Reassembly	One HTTP object may span multiple TCP segments.

Checkpoint	Expected Observation
Browser behaviour	Browser parses HTML and fetches embedded resources.
curl behaviour	curl normally retrieves only the explicitly requested URL.

Command Reference Summary

Command	Purpose
http.request	List requested HTTP objects.
http.response	List HTTP responses.
tcp.len	TCP payload length field.
File > Export Objects > HTTP	Recover transferred files.
tshark --export-objects http,DIR	Command-line HTTP object extraction.
sha256sum	Verify recovered object integrity.

Academic Integrity and Authorization Statement

Student Declaration
I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.